# Introduction

This dataset was scraped from nextspaceflight.com and includes all the space missions since the beginning of Space Race between the USA and the Soviet Union in 1957!

## Install Package with Country Codes

```
pip install iso3166
```

```
Requirement already satisfied: iso3166 in
/usr/local/lib/python3.12/dist-packages (2.1.1)
```

## Upgrade Plotly

Run the cell below if you are working with Google Colab.

```
pip install --upgrade plotly
```

```
Requirement already satisfied: plotly in
/usr/local/lib/python3.12/dist-packages (6.3.1)
Requirement already satisfied: narwhals>=1.15.1 in
/usr/local/lib/python3.12/dist-packages (from plotly) (2.10.0)
Requirement already satisfied: packaging in
/usr/local/lib/python3.12/dist-packages (from plotly) (25.0)
```

## Import Statements

```
import numpy as np
import pandas as pd
import plotly.express as px
import matplotlib.pyplot as plt
import seaborn as sns


from iso3166 import countries
from datetime import datetime, timedelta
```

## Notebook Presentation

```
pd.options.display.float_format = '{:,.2f}'.format
```

## Load the Data

```
df_data = pd.read_csv('mission_launches.csv')
check_ = pd.read_csv('/content/mission_launches.csv')
```

# Preliminary Data Exploration

- What is the shape of `df_data`?
- How many rows and columns does it have?
- What are the column names?
- Are there any NaN values or duplicates?

```
df_data.shape

df_data.columns

df_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4324 entries, 0 to 4323
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Unnamed: 0.1    4324 non-null   int64
 1   Unnamed: 0      4324 non-null   int64
 2   Organisation    4324 non-null   object
 3   Location        4324 non-null   object
 4   Date            4324 non-null   object
 5   Detail          4324 non-null   object
 6   Rocket_Status   4324 non-null   object
 7   Price           964 non-null    object
 8   Mission_Status  4324 non-null   object
dtypes: int64(2), object(7)
memory usage: 304.2+ KB
```

## Data Cleaning - Check for Missing Values and Duplicates

Consider removing columns containing junk data.

```
df_data.isna().sum()

df_data.duplicated()

df_data.dropna(inplace=True)
```

## Descriptive Statistics

```
df_data.describe()
```

{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 8,\n  \"fields\": [\n  {\n  \"column\": \"Unnamed: 0.1\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 1236.7321483922074,\n  \"min\": 0.0,\n  \"max\": 4020.0,\n  \"num_unique_values\": 8,\n  \"samples\": [\n  858.4854771784233,\n  660.5,\n  964.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Unnamed: 0\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 1236.7321483922074,\n  \"min\": 0.0,\n  \"max\": 4020.0,\n  \"num_unique_values\": 8,\n  \"samples\": [\n  858.4854771784233,\n  660.5,\n  964.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe"}

df_data

{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n  \"fields\": [\n  {\n  \"column\": \"Unnamed: 0.1\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Unnamed: 0\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Organisation\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 25,\n  \"samples\": [\n  \"Virgin Orbit\",\n  \"Kosmotras\",\n  \"SpaceX\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Location\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 56,\n  \"samples\": [\n  \"LC-39A, Kennedy Space Center, Florida, USA\",\n  \"Site 31/6, Baikonur Cosmodrome, Kazakhstan\",\n  \"SLC-3E, Vandenberg AFB, California, USA\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Date\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 963,\n  \"samples\": [\n  \"Fri May 21, 2010 22:01 UTC\",\n  \"Fri Jan 15, 2016 16:57 UTC\",\n  \"Wed Sep 30, 2015 20:30 UTC\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Detail\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 962,\n  \"samples\": [\n  \"PSLV-CA | TeLEOS-1 & Rideshares\",\n  \"GSLV Mk III | Demo Flight (CARE)\",\n  \"Atlas V 421 | Morelos-3\"\n  ],\n

```
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Rocket_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n
\"StatusRetired\",\n          \"StatusActive\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\
n        \"dtype\": \"category\",\n        \"num_unique_values\": 56,\
n        \"samples\": [\n          \"50.0\",\n          \"48.5\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Mission_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n
\"Failure\",\n          \"Prelaunch Failure\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"df_data"}
```


# Number of Launches per Company

- List item
- List item

Create a chart that shows the number of space mission launches by organisation.

```
number_launches_by_company =
df_data.groupby("Organisation",as_index=False).agg({'Detail':pd.Series
.count})

query_1 = pd.DataFrame(number_launches_by_company)


query_1

plt.figure(figsize=(14,8))

fig = px.bar(query_1 , x='Organisation',y='Detail',title='Number of
Launches per Company',color='Detail',labels='COUNT')

fig.update_layout(yaxis_title='Count',coloraxis_showscale = False)
```

```
<Figure size 1400x800 with 0 Axes>
```

```
df_data
```


```
{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n
\"fields\": [\n    {\n      \"column\": \"Unnamed: 0.1\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
784,\n        \"min\": 0,\n        \"max\": 4020,\n
\"num_unique_values\": 964,\n        \"samples\": [\n          1253,\n
```

```
1877,\n          3595\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Unnamed: 0\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 784,\n        \"min\": 0,\n
\"max\": 4020,\n        \"num_unique_values\": 964,\n
\"samples\": [\n          1253,\n          1877,\n          3595\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Organisation\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 25,\n        \"samples\": [\n          \"Virgin
Orbit\",\n          \"Kosmotras\",\n          \"SpaceX\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Location\",\n      \"properties\":
{\n        \"dtype\": \"category\",\n        \"num_unique_values\":
56,\n        \"samples\": [\n          \"LC-39A, Kennedy Space Center,
Florida, USA\",\n          \"Site 31/6, Baikonur Cosmodrome,
Kazakhstan\",\n          \"SLC-3E, Vandenberg AFB, California, USA\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Date\",\n      \"properties\":
{\n        \"dtype\": \"string\",\n        \"num_unique_values\":
963,\n        \"samples\": [\n          \"Fri May 21, 2010 22:01
UTC\",\n          \"Fri Jan 15, 2016 16:57 UTC\",\n          \"Wed Sep
30, 2015 20:30 UTC\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Detail\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 962,\n        \"samples\": [\n          \"PSLV-
CA | TeLEOS-1 & Rideshares\",\n          \"GSLV Mk III | Demo Flight
(CARE)\",\n          \"Atlas V 421 | Morelos-3\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Rocket_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n
\"StatusRetired\",\n          \"StatusActive\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\
n        \"dtype\": \"category\",\n        \"num_unique_values\": 56,\
n        \"samples\": [\n          \"50.0\",\n          \"48.5\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Mission_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n
\"Failure\",\n          \"Prelaunch Failure\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"df_data"}
```


# Number of Active versus Retired Rockets

How many rockets are active compared to those that are decomissioned?

```
query_2 = np.where(1,df_data['Rocket_Status'] == 'StatusActive',0)

df_data['Rocket_Status_numerical'] = query_2

px.bar(df_data['Rocket_Status'].value_counts(),color=('red','blue'))

df_data
```


{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n  \"fields\": [\n  {\n  \"column\": \"Unnamed: 0.1\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Unnamed: 0\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Organisation\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 25,\n  \"samples\": [\n  \"Virgin Orbit\",\n  \"Kosmotras\",\n  \"SpaceX\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Location\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 56,\n  \"samples\": [\n  \"LC-39A, Kennedy Space Center, Florida, USA\",\n  \"Site 31/6, Baikonur Cosmodrome, Kazakhstan\",\n  \"SLC-3E, Vandenberg AFB, California, USA\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Date\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 963,\n  \"samples\": [\n  \"Fri May 21, 2010 22:01 UTC\",\n  \"Fri Jan 15, 2016 16:57 UTC\",\n  \"Wed Sep 30, 2015 20:30 UTC\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Detail\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 962,\n  \"samples\": [\n  \"PSLV-CA | TeLEOS-1 & Rideshares\",\n  \"GSLV Mk III | Demo Flight (CARE)\",\n  \"Atlas V 421 | Morelos-3\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Rocket_Status\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 2,\n  \"samples\": [\n  \"StatusRetired\",\n  \"StatusActive\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Price\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 56,\n  \"samples\": [\n  \"50.0\",\n  \"48.5\"\n

```
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Mission_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n
\"Failure\",\n          \"Prelaunch Failure\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Rocket_Status_numerical\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 0,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          0,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df_data"}
```


# Distribution of Mission Status

How many missions were successful? How many missions failed?

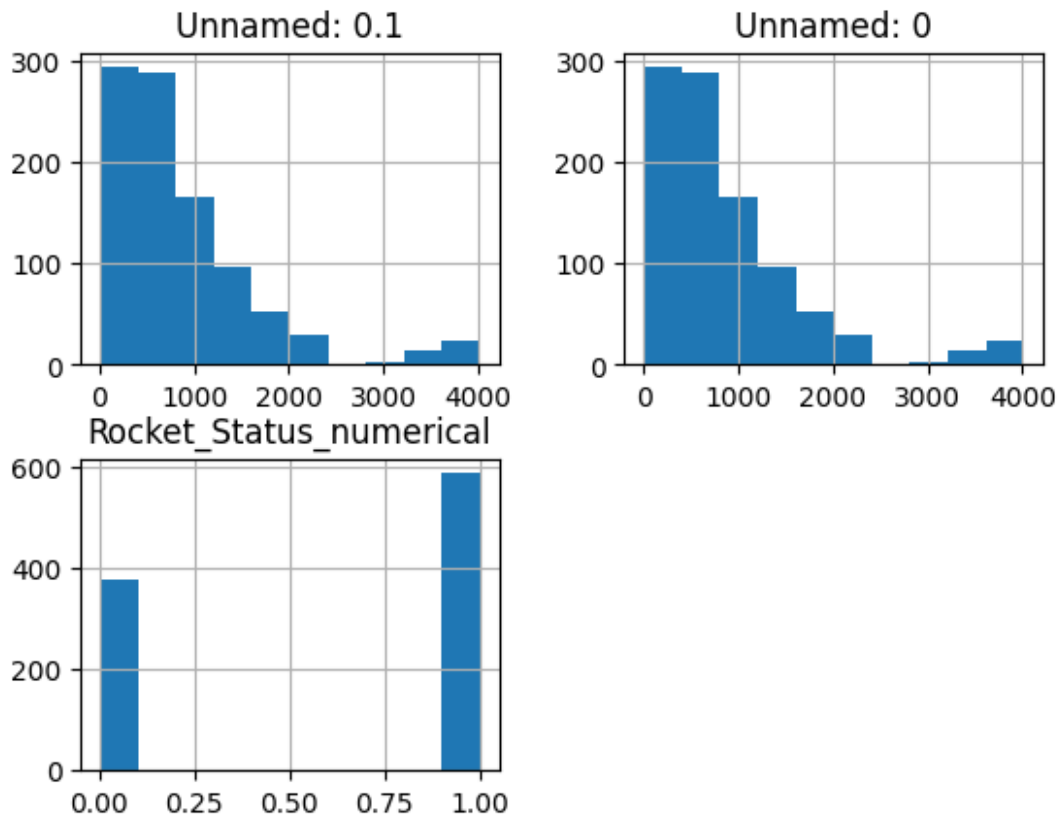
```
px.bar(df_data['Mission_Status'].value_counts(),color=('blue','red','y
ellow','orange'))
```

# How Expensive are the Launches?

Create a histogram and visualise the distribution. The price column is given in USD millions (careful of missing values).

```
df_data.hist()
```

```
array([[<Axes: title={'center': 'Unnamed: 0.1'}>,
        <Axes: title={'center': 'Unnamed: 0'}>],
       [<Axes: title={'center': 'Rocket_Status_numerical'}>, <Axes:
>]],
      dtype=object)
```




# Use a Choropleth Map to Show the Number of Launches by Country

- Create a choropleth map using the plotly documentation
- Experiment with plotly's available colours. I quite like the sequential colour `matter` on this map.
- You'll need to extract a `country` feature as well as change the country names that no longer exist.

Wrangle the Country Names

You'll need to use a 3 letter country code for each country. You might have to change some country names.

- Russia is the Russian Federation
- New Mexico should be USA
- Yellow Sea refers to China
- Shahrud Missile Test Site should be Iran
- Pacific Missile Range Facility should be USA

- Barents Sea should be Russian Federation
- Gran Canaria should be USA

You can use the iso3166 package to convert the country names to Alpha3 format.

```
query_3 = df_data['Location'].str.split(",")
listo = []

for i in df_data['Location']:
  listo.append(i.split(",")[-1].strip())


listo

df_data['country']=listo




df_data['country'].dtype

df_data['country'] = df_data['country'].replace("Russia", "Russian
Federation")
df_data['country'] = df_data['country'].replace("Gran Canaria", "USA")
df_data['country'] = df_data['country'].replace("Yellow Sea", "USA")
df_data['country'] = df_data['country'].replace("Pacific Missile Range
Facility", "USA")

df_data['country'].value_counts()
```

```
country
USA                    491
China                  158
France                  95
India                   67
Russian Federation      54
Kazakhstan              46
Japan                   40
New Zealand             13
Name: count, dtype: int64
```

```
df_data['country'].value_counts()
```

```
country
USA                    491
China                  158
France                  95
India                   67
Russian Federation      54
Kazakhstan              46
```

```
Japan                   40
New Zealand             13
Name: count, dtype: int64
```

```
alpha_3_count = []

for i in df_data['country']:
  alpha_3_count.append(countries.get(i)[2])


df_data['country_alpha_3'] = alpha_3_count


df_data

fig = px.choropleth(locations = df_data['country_alpha_3'],color=[i
for i in range(0,964)])

fig.show()
```

# Use a Choropleth Map to Show the Number of Failures by Country

```
query_5 = df_data[df_data['Mission_Status'] == 'Failure' ]

logo =
query_5.groupby('country_alpha_3',as_index=False).agg({'Mission_Status
':pd.Series.count})


fig_1  = px.choropleth(locations =
logo['country_alpha_3'],color=logo['Mission_Status'],color_continuous_
scale='Reds')

fig_1.show()

df_data
```


```
{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n
\"fields\": [\n  {\n  \"column\": \"Unnamed: 0.1\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
784,\n  \"min\": 0,\n  \"max\": 4020,\n
\"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n
1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Unnamed: 0\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 784,\n  \"min\": 0,\n
\"max\": 4020,\n  \"num_unique_values\": 964,\n
```

\"samples\": [\n          1253,\n          1877,\n          3595\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Organisation\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 25,\n        \"samples\": [\n          \"Virgin
Orbit\",\n          \"Kosmotras\",\n          \"SpaceX\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Location\",\n      \"properties\":
{\n        \"dtype\": \"category\",\n        \"num_unique_values\":
56,\n        \"samples\": [\n          \"LC-39A, Kennedy Space Center,
Florida, USA\",\n          \"Site 31/6, Baikonur Cosmodrome,
Kazakhstan\",\n          \"SLC-3E, Vandenberg AFB, California, USA\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Date\",\n      \"properties\":
{\n        \"dtype\": \"string\",\n        \"num_unique_values\":
963,\n        \"samples\": [\n          \"Fri May 21, 2010 22:01
UTC\",\n          \"Fri Jan 15, 2016 16:57 UTC\",\n          \"Wed Sep
30, 2015 20:30 UTC\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Detail\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 962,\n        \"samples\": [\n          \"PSLV-
CA | TeLEOS-1 & Rideshares\",\n          \"GSLV Mk III | Demo Flight
(CARE)\",\n          \"Atlas V 421 | Morelos-3\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Rocket_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n
\"StatusRetired\",\n          \"StatusActive\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\
n        \"dtype\": \"category\",\n        \"num_unique_values\": 56,\
n        \"samples\": [\n          \"50.0\",\n          \"48.5\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Mission_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n
\"Failure\",\n          \"Prelaunch Failure\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Rocket_Status_numerical\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 0,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          0,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"country\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 8,\n        \"samples\":
[\n          \"China\",\n          \"Russian Federation\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"country_alpha_3\",\n

\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 8,\n        \"samples\": [\n          \"CHN\",\
n          \"RUS\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df_data"}

# Create a Plotly Sunburst Chart of the countries, organisations, and mission status.

```
abc =
df_data.groupby(['country','Organisation','Mission_Status'],as_index=F
alse).size().rename(columns={'size':'Mission_Count'})

fig = px.sunburst(abc ,
path=['country','Organisation','Mission_Status'], color =
'Mission_Status',color_discrete_map ={'Success':'green',

'Failure':'orange',

'Prelaunch Failure':'blue'

},
    title='Missions by Country, Organisation, and Status')

fig.show()
```

# Analyse the Total Amount of Money Spent by Organisation on Space Missions

```
df_data['Price']  = df_data['Price'].str.replace(',',"")

df_data['Price'] = pd.to_numeric(df_data['Price'])


abc1 =
df_data.groupby('Organisation',as_index=False).agg({'Price':pd.Series.
sum})
```

```
px.bar(abc1,x='Organisation',y='Price',color = 'Organisation')
```

```
abc1
```

{"summary":"{\n  \"name\": \"abc1\",\n  \"rows\": 25,\n  \"fields\": [\n  {\n  \"column\": \"Organisation\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 25,\n  \"samples\": [\n  \"ISRO\",\n  \"RVSN USSR\",\n  \"Arianespace\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Price\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 15328.250167845728,\n  \"min\": 12.0,\n  \"max\": 76280.0,\n  \"num_unique_values\": 25,\n  \"samples\": [\n  2177.0,\n  10000.0,\n  16345.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe","variable_name":"abc1"}

```
df_data
```

{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n  \"fields\": [\n  {\n  \"column\": \"Unnamed: 0.1\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Unnamed: 0\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Organisation\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 25,\n  \"samples\": [\n  \"Virgin Orbit\",\n  \"Kosmotras\",\n  \"SpaceX\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Location\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 56,\n  \"samples\": [\n  \"LC-39A, Kennedy Space Center, Florida, USA\",\n  \"Site 31/6, Baikonur Cosmodrome, Kazakhstan\",\n  \"SLC-3E, Vandenberg AFB, California, USA\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Date\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 963,\n  \"samples\": [\n  \"Fri May 21, 2010 22:01 UTC\",\n  \"Fri Jan 15, 2016 16:57 UTC\",\n  \"Wed Sep 30, 2015 20:30 UTC\"\n  ],\n  \"semantic_type\": \"\",\n

\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Detail\",\n      \"properties\": {\n        \"dtype\": \"string\",\n \"num_unique_values\": 962,\n        \"samples\": [\n          \"PSLV-CA | TeLEOS-1 & Rideshares\",\n          \"GSLV Mk III | Demo Flight (CARE)\",\n          \"Atlas V 421 | Morelos-3\"\n        ],\n \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Rocket_Status\",\n \"properties\": {\n        \"dtype\": \"category\",\n \"num_unique_values\": 2,\n        \"samples\": [\n \"StatusRetired\",\n          \"StatusActive\"\n        ],\n \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 288.45073217474595,\n        \"min\": 5.3,\n        \"max\": 5000.0,\n \"num_unique_values\": 56,\n        \"samples\": [\n          50.0,\n 48.5\n        ],\n        \"semantic_type\": \"\",\n \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Mission_Status\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Failure\",\n          \"Prelaunch Failure\"\n        ],\n        \"semantic_type\": \"\",\n \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Rocket_Status_numerical\",\n      \"properties\": {\n \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"China\",\n          \"Russian Federation\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n }\n    },\n    {\n      \"column\": \"country_alpha_3\",\n \"properties\": {\n        \"dtype\": \"category\",\n \"num_unique_values\": 8,\n        \"samples\": [\n          \"CHN\",\n          \"RUS\"\n        ],\n        \"semantic_type\": \"\",\n \"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe","variable_name":"df_data"}

# Analyse the Amount of Money Spent by Organisation per Launch

```
avg_spend_organisation = df_data.groupby('Organisation',as_index=False).agg({'Price':pd.Series.mean})
```

```
px.bar(avg_spend_organisation,x='Organisation',y='Price',color =
'Organisation')



yearly_launches = df_data
```

# Chart the Number of Launches per Year

```
zxc = df_data['Date'].str.split(" ")
bhc = []

for i in zxc:
  bhc.append(int(i[3]))

df_data['Year']= bhc
df_data
```


```
{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n
\"fields\": [\n    {\n      \"column\": \"Unnamed: 0.1\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
784,\n        \"min\": 0,\n        \"max\": 4020,\n
\"num_unique_values\": 964,\n        \"samples\": [\n          1253,\n
1877,\n          3595\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Unnamed: 0\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 784,\n        \"min\": 0,\n
\"max\": 4020,\n        \"num_unique_values\": 964,\n
\"samples\": [\n          1253,\n          1877,\n          3595\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Organisation\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 25,\n        \"samples\": [\n          \"Virgin
Orbit\",\n          \"Kosmotras\",\n          \"SpaceX\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Location\",\n      \"properties\":
{\n        \"dtype\": \"category\",\n        \"num_unique_values\":
56,\n        \"samples\": [\n          \"LC-39A, Kennedy Space Center,
Florida, USA\",\n          \"Site 31/6, Baikonur Cosmodrome,
Kazakhstan\",\n          \"SLC-3E, Vandenberg AFB, California, USA\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Date\",\n      \"properties\":
{\n        \"dtype\": \"string\",\n        \"num_unique_values\":
963,\n        \"samples\": [\n          \"Fri May 21, 2010 22:01
UTC\",\n          \"Fri Jan 15, 2016 16:57 UTC\",\n          \"Wed Sep
30, 2015 20:30 UTC\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Detail\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
```

\"num_unique_values\": 962,\n        \"samples\": [\n          \"PSLV-CA | TeLEOS-1 & Rideshares\",\n          \"GSLV Mk III | Demo Flight (CARE)\",\n          \"Atlas V 421 | Morelos-3\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Rocket_Status\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"StatusRetired\",\n          \"StatusActive\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 288.45073217474595,\n        \"min\": 5.3,\n        \"max\": 5000.0,\n        \"num_unique_values\": 56,\n        \"samples\": [\n          50.0,\n          48.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Mission_Status\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Failure\",\n          \"Prelaunch Failure\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Rocket_Status_numerical\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"China\",\n          \"Russian Federation\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"country_alpha_3\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"CHN\",\n          \"RUS\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Year\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 12,\n        \"min\": 1964,\n        \"max\": 2020,\n        \"num_unique_values\": 50,\n        \"samples\": [\n          2007,\n          1981\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe","variable_name":"df_data"}

```
uio =
df_data.groupby('Year',as_index=False).agg({'Location':pd.Series.count
})

uio = pd.DataFrame(uio)

uio
```

{"summary":"{\n  \"name\": \"uio\",\n  \"rows\": 50,\n  \"fields\": [\n  {\n  \"column\": \"Year\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 16,\n  \"min\": 1964,\n  \"max\": 2020,\n  \"num_unique_values\": 50,\n  \"samples\": [\n  1984,\n  2010,\n  2001\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Location\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 19,\n  \"min\": 1,\n  \"max\": 88,\n  \"num_unique_values\": 30,\n  \"samples\": [\n  88,\n  20,\n  41\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe","variable_name":"uio"}

```
uio.rename(columns={'Location':"Count"},inplace=True)

px.line(uio,x='Year',y='Count')
```

# Chart the Number of Launches Month-on-Month until the Present

Which month has seen the highest number of launches in all time? Superimpose a rolling average on the month on month time series chart.

```
df_data
```

{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n  \"fields\": [\n  {\n  \"column\": \"Unnamed: 0.1\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Unnamed: 0\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 784,\n  \"min\": 0,\n  \"max\": 4020,\n  \"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n  1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Organisation\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\": 25,\n  \"samples\": [\n  \"Virgin Orbit\",\n  \"Kosmotras\",\n  \"SpaceX\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Location\",\n  \"properties\": {\n  \"dtype\": \"category\",\n  \"num_unique_values\":

56,\n        \"samples\": [\n          \"LC-39A, Kennedy Space Center, Florida, USA\",\n          \"Site 31/6, Baikonur Cosmodrome, Kazakhstan\",\n          \"SLC-3E, Vandenberg AFB, California, USA\"\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 963,\n        \"samples\": [\n          \"Fri May 21, 2010 22:01 UTC\",\n          \"Fri Jan 15, 2016 16:57 UTC\",\n          \"Wed Sep 30, 2015 20:30 UTC\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Detail\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 962,\n        \"samples\": [\n          \"PSLV-CA | TeLEOS-1 & Rideshares\",\n          \"GSLV Mk III | Demo Flight (CARE)\",\n          \"Atlas V 421 | Morelos-3\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Rocket_Status\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"StatusRetired\",\n          \"StatusActive\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 288.45073217474595,\n        \"min\": 5.3,\n        \"max\": 5000.0,\n        \"num_unique_values\": 56,\n        \"samples\": [\n          50.0,\n          48.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Mission_Status\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Failure\",\n          \"Prelaunch Failure\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Rocket_Status_numerical\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"China\",\n          \"Russian Federation\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"country_alpha_3\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"CHN\",\n          \"RUS\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Year\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 12,\n        \"min\": 1964,\n        \"max\": 2020,\n        \"num_unique_values\": 50,\n        \"samples\": [\n          2007,\n          1981\n        ],\n        \"semantic_type\": \"\",\n

\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df_data"}

```
from datetime import datetime

bhce = []
zhc = df_data['Date'].str.split()

for i in zhc:
  day_month_year = f"{i[1]}"


  bhce.append(datetime.strptime(day_month_year,"%b").strftime("%b"))

print(bhce)
df_data['Month'] = bhce

df_data



monthly_group =
df_data.groupby('Month').size().reset_index(name='Launches')

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

monthly_group['Month'] = pd.Categorical(monthly_group['Month'],
categories=months, ordered=True)

monthly_group=monthly_group.sort_values('Month')

# acd =
df_data.groupby('Month_Year',as_index=False).agg({'Location':pd.Series
.count})

# acd.sort_values('Location',ascending=False).head(1)
```

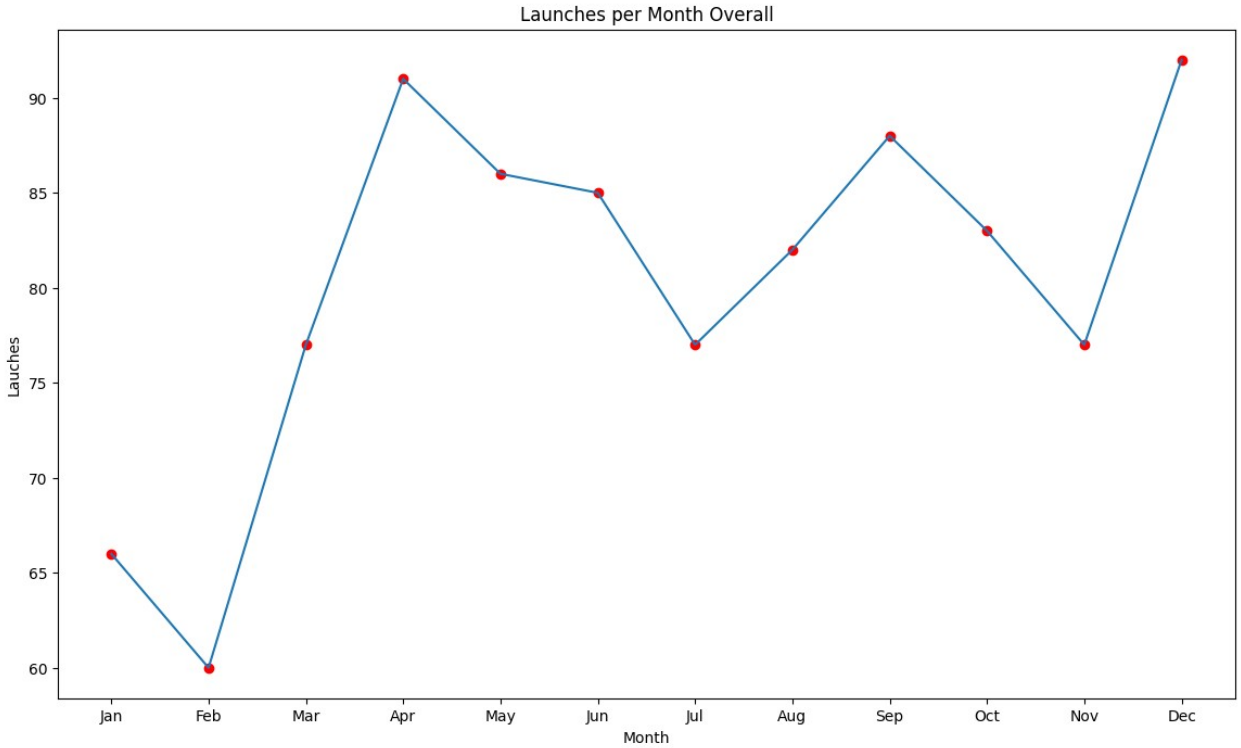
```
['Aug', 'Aug', 'Jul', 'Jul', 'Jul', 'Jul', 'Jul', 'Jul', 'Jul', 'Jul',
'Jul', 'Jul', 'Jul', 'Jul', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun',
'Jun', 'May', 'May', 'May', 'May', 'May', 'May', 'Apr', 'Apr', 'Apr',
'Apr', 'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Feb', 'Feb', 'Feb',
'Feb', 'Feb', 'Feb', 'Feb', 'Feb', 'Jan', 'Jan', 'Jan', 'Jan', 'Jan',
'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec',
'Dec', 'Nov', 'Nov', 'Nov', 'Nov', 'Nov', 'Nov', 'Nov', 'Oct', 'Oct',
'Oct', 'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Sep', 'Aug', 'Aug',
'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Jul', 'Jul',
'Jul', 'Jul', 'Jul', 'Jul', 'Jul', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun',
'Jun', 'May', 'May', 'May', 'May', 'May', 'May', 'Apr', 'Apr', 'Apr',
'Apr', 'Apr', 'Apr', 'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Feb', 'Feb',
'Jan', 'Jan', 'Jan', 'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec',
```

```
'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Nov', 'Nov', 'Nov', 'Nov',
'Nov', 'Nov', 'Nov', 'Nov', 'Nov', 'Nov', 'Oct', 'Oct', 'Oct', 'Oct',
'Oct', 'Oct', 'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Sep', 'Aug',
'Aug', 'Aug', 'Jul', 'Jul', 'Jul', 'Jul', 'Jul', 'Jul', 'Jun', 'Jun',
'Jun', 'Jun', 'Jun', 'Jun', 'May', 'May', 'May', 'May', 'May', 'May',
'May', 'Apr', 'Apr', 'Apr', 'Apr', 'Apr', 'Apr', 'Apr', 'Apr', 'Apr',
'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Feb', 'Feb', 'Feb', 'Feb',
'Feb', 'Feb', 'Jan', 'Jan', 'Jan', 'Jan', 'Jan', 'Jan', 'Jan', 'Jan',
'Jan', 'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec',
'Dec', 'Nov', 'Nov', 'Nov', 'Nov', 'Oct', 'Oct', 'Oct', 'Oct', 'Oct',
'Oct', 'Oct', 'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Sep', 'Sep',
'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Jul', 'Jul', 'Jun',
'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun',
'Jun', 'May', 'May', 'May', 'May', 'May', 'Apr', 'Apr', 'Mar', 'Mar',
'Mar', 'Mar', 'Mar', 'Feb', 'Feb', 'Feb', 'Jan', 'Jan', 'Jan', 'Dec',
'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Nov', 'Nov',
'Nov', 'Nov', 'Nov', 'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Sep',
'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Aug', 'Jul', 'Jul', 'Jun',
'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'May', 'May', 'May',
'May', 'May', 'Apr', 'Apr', 'Apr', 'Mar', 'Mar', 'Mar', 'Mar', 'Mar',
'Mar', 'Mar', 'Mar', 'Mar', 'Feb', 'Feb', 'Feb', 'Feb', 'Jan', 'Jan',
'Jan', 'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Dec', 'Nov', 'Nov', 'Oct',
'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Sep', 'Aug', 'Aug', 'Aug',
'Jul', 'Jul', 'Jul', 'Jun', 'Jun', 'May', 'May', 'Apr', 'Apr', 'Apr',
'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Mar', 'Feb', 'Feb', 'Feb',
'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Dec', 'Dec', 'Nov', 'Oct', 'Oct',
'Oct', 'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Aug', 'Aug', 'Aug',
'Jul', 'Jul', 'Jul', 'Jul', 'Jun', 'Jun', 'May', 'May', 'May', 'May',
'Apr', 'Apr', 'Apr', 'Apr', 'Apr', 'Mar', 'Feb', 'Feb', 'Feb', 'Jan',
'Jan', 'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Nov', 'Nov', 'Nov', 'Nov',
'Nov', 'Sep', 'Sep', 'Sep', 'Sep', 'Sep', 'Aug', 'Aug', 'Aug', 'Aug',
'Jul', 'Jul', 'Jul', 'Jul', 'Jun', 'May', 'May', 'Apr', 'Mar', 'Mar',
'Feb', 'Feb', 'Feb', 'Jan', 'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Nov',
'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Aug', 'Aug', 'Jul', 'Jul', 'Jul',
'Jun', 'Jun', 'Jun', 'May', 'May', 'May', 'May', 'Apr', 'Apr', 'Feb',
'Feb', 'Dec', 'Dec', 'Nov', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Aug',
'Aug', 'Aug', 'Jul', 'Jul', 'Jul', 'Jun', 'May', 'May', 'May', 'Apr',
'Apr', 'Apr', 'Mar', 'Mar', 'Mar', 'Feb', 'Feb', 'Feb', 'Jan', 'Jan',
'Dec', 'Dec', 'Dec', 'Nov', 'Nov', 'Nov', 'Oct', 'Sep', 'Sep', 'Sep',
'Aug', 'Aug', 'Jul', 'Jun', 'Jun', 'Jun', 'Jun', 'Jun', 'May', 'May',
'May', 'May', 'Apr', 'Apr', 'Apr', 'Apr', 'Mar', 'Feb', 'Feb', 'Nov',
'Nov', 'Nov', 'Nov', 'Oct', 'Oct', 'Oct', 'Oct', 'Sep', 'Sep', 'Sep',
'Aug', 'Aug', 'Jul', 'Jul', 'Jul', 'Jul', 'Jul', 'Jun', 'Jun', 'May',
'May', 'May', 'Apr', 'Apr', 'Mar', 'Mar', 'Feb', 'Feb', 'Jan', 'Jan',
'Dec', 'Nov', 'Nov', 'Nov', 'Oct', 'Oct', 'Oct', 'Oct', 'Oct', 'Sep',
'Aug', 'Aug', 'Aug', 'Jul', 'Jun', 'May', 'May', 'May', 'Apr', 'Apr',
'Apr', 'Apr', 'Apr', 'Mar', 'Mar', 'Feb', 'Feb', 'Jan', 'Dec', 'Nov',
'Nov', 'Nov', 'Oct', 'Oct', 'Sep', 'Sep', 'Aug', 'Aug', 'Jun', 'Jun',
'Jun', 'Jun', 'May', 'May', 'May', 'May', 'Apr', 'Apr', 'Apr', 'Apr',
```

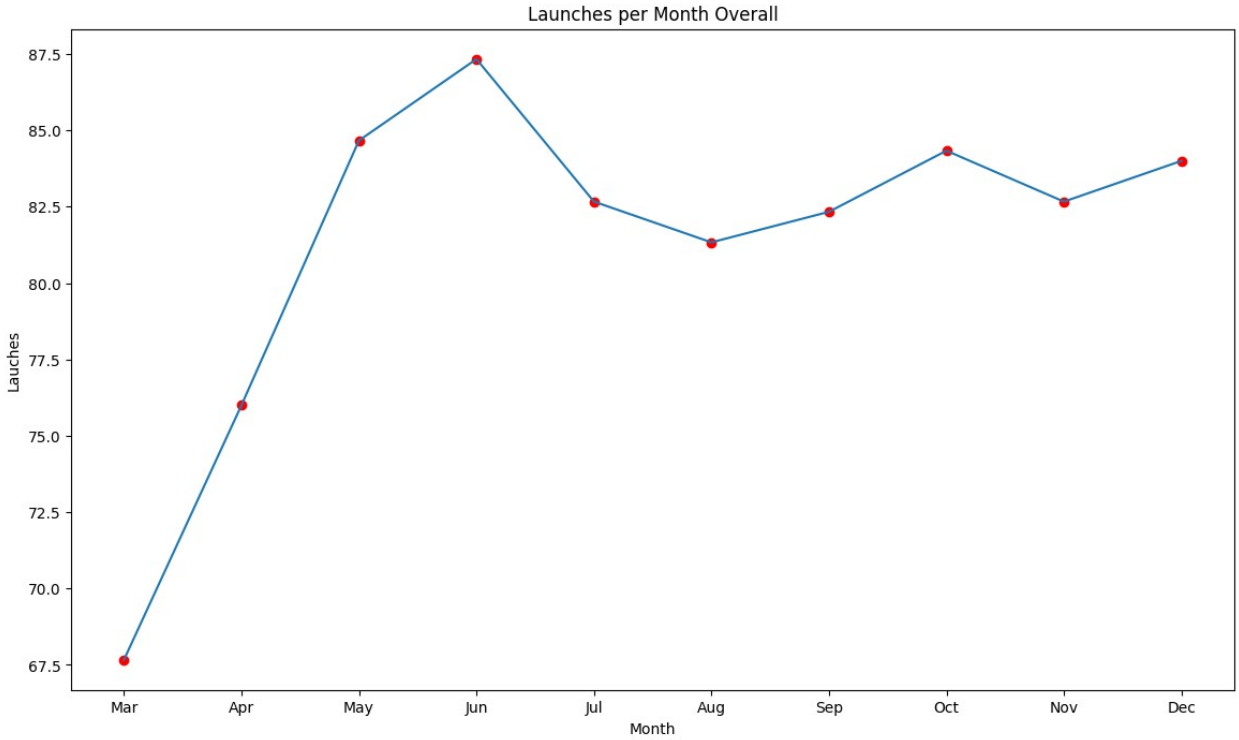
```
'Apr', 'Apr', 'Mar', 'Mar', 'Mar', 'Feb', 'Jan', 'Dec', 'Dec', 'Dec',
'Dec', 'Nov', 'Oct', 'Oct', 'Sep', 'Sep', 'Sep', 'Sep', 'Aug', 'Jul',
'Jul', 'Jul', 'Jul', 'Jul', 'Jun', 'May', 'May', 'Apr', 'Apr', 'Apr',
'Mar', 'Mar', 'Mar', 'Jan', 'Nov', 'Oct', 'Sep', 'Aug', 'Aug', 'Aug',
'Aug', 'Aug', 'Jul', 'Jul', 'May', 'Apr', 'Apr', 'Mar', 'Feb', 'Dec',
'Dec', 'Dec', 'Nov', 'Nov', 'Nov', 'Oct', 'Sep', 'Sep', 'Sep', 'Aug',
'Jul', 'Jul', 'Jun', 'May', 'Apr', 'Mar', 'Dec', 'Nov', 'Nov', 'Oct',
'Oct', 'Oct', 'Oct', 'Aug', 'Aug', 'Jul', 'Jun', 'Jun', 'May', 'May',
'May', 'Apr', 'Mar', 'Jan', 'Jan', 'Jan', 'Dec', 'Dec', 'Dec', 'Nov',
'Nov', 'Oct', 'Oct', 'Sep', 'Aug', 'Jun', 'Jun', 'Jun', 'May', 'Apr',
'Mar', 'Mar', 'Feb', 'Dec', 'Oct', 'Sep', 'Aug', 'Aug', 'Jul', 'Apr',
'Apr', 'Mar', 'Feb', 'Dec', 'Nov', 'Oct', 'Oct', 'Oct', 'Sep', 'Sep',
'Sep', 'Jul', 'Jun', 'May', 'May', 'Mar', 'Feb', 'Jan', 'Jan', 'Dec',
'Dec', 'Dec', 'Dec', 'Oct', 'Jul', 'Jun', 'Jun', 'May', 'May', 'May',
'May', 'Apr', 'Mar', 'Dec', 'Dec', 'Dec', 'Oct', 'Oct', 'Oct', 'Sep',
'Aug', 'Aug', 'Jun', 'May', 'May', 'Apr', 'Apr', 'Mar', 'Feb', 'Feb',
'Jan', 'Dec', 'Dec', 'Nov', 'Oct', 'Sep', 'Sep', 'Sep', 'Aug', 'Aug',
'Aug', 'Jul', 'May', 'May', 'Apr', 'Apr', 'Apr', 'Feb', 'Jan', 'Dec',
'Nov', 'Oct', 'Sep', 'Aug', 'Jul', 'Jun', 'May', 'May', 'Mar', 'Mar',
'Mar', 'Feb', 'Jan', 'Nov', 'Oct', 'Oct', 'Sep', 'Jul', 'Jun', 'Jun',
'Apr', 'Mar', 'Feb', 'Nov', 'Nov', 'Oct', 'Sep', 'Sep', 'Sep', 'Jul',
'Jul', 'Jun', 'May', 'Apr', 'Mar', 'Mar', 'Feb', 'Feb', 'Jan', 'Dec',
'Oct', 'Oct', 'Oct', 'Sep', 'Sep', 'Jun', 'Apr', 'Apr', 'Apr', 'Feb',
'Jan', 'Dec', 'Oct', 'Oct', 'Sep', 'Sep', 'Aug', 'Jul', 'Jun', 'May',
'Apr', 'Mar', 'Jan', 'Nov', 'Sep', 'Aug', 'Jul', 'Jun', 'Apr', 'Apr',
'Dec', 'Nov', 'Oct', 'Oct', 'Jun', 'Apr', 'Apr', 'Mar', 'Feb', 'Jan',
'Jan', 'Nov', 'Oct', 'Sep', 'Aug', 'May', 'Mar', 'Dec', 'Nov', 'Sep',
'Sep', 'Aug', 'Sep', 'Aug', 'May', 'Oct', 'Jan', 'Jan', 'Nov', 'Nov',
'Oct', 'Oct', 'Aug', 'Jul', 'Jun', 'May', 'Apr', 'Jan', 'Nov', 'Oct',
'Sep', 'Aug', 'Apr', 'Feb', 'Nov', 'Sep', 'Aug', 'Jun', 'Apr', 'Nov',
'Sep', 'Jun', 'Mar', 'Nov', 'Apr', 'May', 'Dec', 'Apr', 'Jul', 'Jan',
'Apr', 'Nov', 'Jul', 'Jun', 'May', 'Apr', 'Mar', 'Mar', 'Jan', 'Dec',
'Dec', 'Nov', 'Aug', 'Aug', 'Jun', 'Apr', 'Apr', 'Mar', 'Jan', 'Dec',
'Nov', 'Oct', 'Sep', 'Aug', 'Jun', 'Apr', 'Feb', 'Dec', 'Sep', 'Jul',
'May', 'Feb', 'Dec', 'Sep']
```

```
plt.figure(figsize=(14,8))

plt.plot(monthly_group['Month'],monthly_group['Launches'],'ro')
plt.plot(monthly_group['Month'],monthly_group['Launches'])
plt.title("Launches per Month Overall")

plt.xlabel('Month')
plt.ylabel("Lauches")
```

```
Text(0, 0.5, 'Lauches')
```




```
rolling_months = monthly_group['Launches'].rolling(window=3).mean()

plt.figure(figsize=(14,8))

plt.plot(monthly_group['Month'],rolling_months,'ro')
plt.plot(monthly_group['Month'],rolling_months)
plt.title("Launches per Month Overall")

plt.xlabel('Month')
plt.ylabel("Lauches")

Text(0, 0.5, 'Lauches')
```




# Launches per Month: Which months are most popular and least popular for launches?

Some months have better weather than others. Which time of year seems to be best for space missions?

```
most_popular = monthly_group['Launches'].max()
least_popular = monthly_group['Launches'].min()

month_pop = monthly_group[monthly_group['Launches']==most_popular]
month_least = monthly_group[monthly_group['Launches']==least_popular]

print(f"THe most popular month is
{monthly_group[monthly_group['Launches']==monthly_group['Launches'].ma
x()].iloc[0]['Month']} and the least popular month is
{monthly_group[monthly_group['Launches']==monthly_group['Launches'].mi
n()].iloc[0]['Month']}")
```

```
THe most popular month is Dec and the least popular month is Feb
```

# How has the Launch Price varied Over Time?

Create a line chart that shows the average price of rocket launches over time.

```
yearly_average =
df_data.groupby('Year',as_index=False).agg({'Price':pd.Series.mean})

plt.figure(figsize=(14,8))
px.line(yearly_average,x='Year',y='Price',title='Average launch price
per year')
```

```
<Figure size 1400x800 with 0 Axes>
```

# Chart the Number of Launches over Time by the Top 10 Organisations.

How has the dominance of launches changed over time between the different players?

```
yearly_average =
df_data.groupby(['Year','Organisation'],as_index=False).size().rename(
columns={'size':'Launches'})
yearly_average
```


```
{"summary":"{\n  \"name\": \"yearly_average\",\n  \"rows\": 268,\n
\"fields\": [\n  {\n  \"column\": \"Year\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
12,\n  \"min\": 1964,\n  \"max\": 2020,\n
\"num_unique_values\": 50,\n  \"samples\": [\n  1984,\n
2010,\n  2001\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Organisation\",\n  \"properties\": {\n  \"dtype\":
\"category\",\n  \"num_unique_values\": 25,\n
\"samples\": [\n  \"Lockheed\",\n  \"SpaceX\",\n
\"US Air Force\"\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Launches\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 3,\n  \"min\": 1,\n
\"max\": 27,\n  \"num_unique_values\": 17,\n  \"samples\":
[\n  2,\n  3,\n  4\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  }\n  ]\n}","type":"dataframe","variable_name":"yearly_average"}
```


```
px.line(yearly_average,x='Year',y='Launches',color='Organisation')
```

# Cold War Space Race: USA vs USSR

The cold war lasted from the start of the dataset up until 1991.

```
USA_VS_RUSSIA = df_data[(df_data['country'].isin(['Russian
Federation']))]

USA_VS_RUSSIA
```

{"repr_error":"0","type":"dataframe","variable_name":"USA_VS_RUSSIA"}

```
USA_RUSSIA = check_[(check_['Location'].str.contains('Kazakhstan')) |
(check_['Location'].str.contains('Russia')) |
(check_['Location'].str.contains('USA'))].copy()

zxc1 = USA_RUSSIA['Date'].str.split(" ")



bhc1 = []

for i in zxc1:
  bhc1.append(int(i[3]))

USA_RUSSIA['Year']= bhc1



USA_RUSSIA.dtypes
```

```
Unnamed: 0.1       int64
Unnamed: 0         int64
Organisation      object
Location          object
Date              object
Detail            object
Rocket_Status     object
Price             object
Mission_Status    object
Year               int64
dtype: object
```

```
USA_RUSSIA=USA_RUSSIA[USA_RUSSIA['Year'] < 1991]




listo = []

for i in USA_RUSSIA['Location']:
```

```
  listo.append(i.split(",")[-1].strip())




USA_RUSSIA['country']=listo

USA_RUSSIA


grp1 =
USA_RUSSIA.groupby(['Year','country'],as_index=False).size().rename(co
lumns={'size':'Launch'})



px.line(grp1,x='Year',y='Launch',color='country')

listo = []
for i in check_['Location']:
  listo.append(i.split(",")[-1].strip())




check_['country']=listo

check_['country'].unique()

array(['USA', 'China', 'Kazakhstan', 'Japan', 'Israel', 'New Zealand',
       'Russia', 'Shahrud Missile Test Site', 'France', 'Iran',
'India',
       'New Mexico', 'Yellow Sea', 'North Korea',
       'Pacific Missile Range Facility', 'Pacific Ocean', 'South
Korea',
       'Barents Sea', 'Brazil', 'Gran Canaria', 'Kenya', 'Australia'],
      dtype=object)
```

## Create a Plotly Pie Chart comparing the total number of launches of the USSR and the USA

Hint: Remember to include former Soviet Republics like Kazakhstan when analysing the total number of launches.

```
grp1['country'] = grp1['country'].replace('Kazakhstan','Russia')

px.pie(grp1,names='country',values='Launch')
```

## Create a Chart that Shows the Total Number of Launches Year-On-Year by the Two Superpowers

```
# already done


check_ = pd.read_csv('/content/mission_launches.csv')

bhc1 = []

for i in check_['Date'].str.split(" "):
  bhc1.append(int(i[3]))

check_['Year']= bhc1
```

## Chart the Total Number of Mission Failures Year on Year.

```
jogo= check_[check_['Mission_Status'] == 'Failure']


yearly_failue =
jogo.groupby(['Year','Mission_Status'],as_index=False).size()
yearly_failue

fig = px.line(yearly_failue,x='Year',y='size')

fig.update_xaxes(range=[1957,2018])




yearly_status= check_.groupby('Year',as_index=False).size()

yearly_status['Failure_count'] = yearly_failue['size']

failure_perc = yearly_status['Failure_count']/yearly_status['size']

yearly_status['failure_perc']= failure_perc


yearly_status
```

{"summary":"{\n  \"name\": \"yearly_status\",\n  \"rows\": 64,\n  \"fields\": [\n  {\n  \"column\": \"Year\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 18,\n  \"min\": 1957,\n  \"max\": 2020,\n  \"num_unique_values\": 64,\n  \"samples\": [\n  2009,\n  2015,\n  1957\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\":

\"size\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 26,\n        \"min\": 3,\n        \"max\": 119,\n
\"num_unique_values\": 49,\n        \"samples\": [\n          119,\n
92,\n          109\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Failure_count\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 4,\n        \"min\": 1,\n
\"max\": 20,\n        \"num_unique_values\": 14,\n        \"samples\":
[\n          7,\n          4,\n          1\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"failure_perc\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.12484637017349134,\n        \"min\": 0.010309278350515464,\n
\"max\": 0.7142857142857143,\n        \"num_unique_values\": 61,\n
\"samples\": [\n          0.3333333333333333,\n
0.18292682926829268,\n          0.025\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"yearly_status"}

## Chart the Percentage of Failures over Time

Did failures go up or down over time? Did the countries get better at minimising risk and improving their chances of success over time?

```
px.line(yearly_status,x='Year',y='failure_perc')
```

# For Every Year Show which Country was in the Lead in terms of Total Number of Launches up to and including including 2020)

Do the results change if we only look at the number of successful launches?

```
listo = []

for i in check_['Location']:
  listo.append(i.split(",")[-1].strip())



check_['country']=listo
```

```
yearly_country_launches =
check_.groupby(['Year','country'],as_index=True).size().reset_index(na
me = 'Launches')

yearly_country_launches
```


```
{"summary":"{\n  \"name\": \"yearly_country_launches\",\n  \"rows\":
436,\n  \"fields\": [\n  {\n  \"column\": \"Year\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
17,\n  \"min\": 1957,\n  \"max\": 2020,\n
\"num_unique_values\": 64,\n  \"samples\": [\n  2009,\n
2015,\n  1957\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"country\",\n  \"properties\": {\n  \"dtype\":
\"category\",\n  \"num_unique_values\": 22,\n
\"samples\": [\n  \"Kazakhstan\",\n  \"North Korea\",\
n  \"India\"\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Launches\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 13,\n  \"min\": 1,\n
\"max\": 72,\n  \"num_unique_values\": 50,\n  \"samples\":
[\n  17,\n  21,\n  3\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  }\n  ]\
n}","type":"dataframe","variable_name":"yearly_country_launches"}
```


```
px.line(yearly_country_launches,x='Year',y='Launches',color='country')
```

# Create a Year-on-Year Chart Showing the Organisation Doing the Most Number of Launches

Which organisation was dominant in the 1970s and 1980s? Which organisation was dominant in 2018, 2019 and 2020?

```
organisation_country_launches =
check_.groupby(['Year','Organisation'],as_index=True).size().reset_ind
ex(name = 'Launches')

organisation_country_launches

px.line(organisation_country_launches,x='Year',y='Launches',color='Org
anisation')
```

```
df_data

rel_df =
df_data[['Organisation','Price','Rocket_Status_numerical','country_alp
ha_3','Year','Month','Mission_Status']]


df_data
```


```
{"summary":"{\n  \"name\": \"df_data\",\n  \"rows\": 964,\n
\"fields\": [\n  {\n  \"column\": \"Unnamed: 0.1\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
784,\n  \"min\": 0,\n  \"max\": 4020,\n
\"num_unique_values\": 964,\n  \"samples\": [\n  1253,\n
1877,\n  3595\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Unnamed: 0\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 784,\n  \"min\": 0,\n
\"max\": 4020,\n  \"num_unique_values\": 964,\n
\"samples\": [\n  1253,\n  1877,\n  3595\n
],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n
}\n  },\n  {\n  \"column\": \"Organisation\",\n
\"properties\": {\n  \"dtype\": \"category\",\n
\"num_unique_values\": 25,\n  \"samples\": [\n  \"Virgin
Orbit\",\n  \"Kosmotras\",\n  \"SpaceX\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Location\",\n  \"properties\":
{\n  \"dtype\": \"category\",\n  \"num_unique_values\":
56,\n  \"samples\": [\n  \"LC-39A, Kennedy Space Center,
Florida, USA\",\n  \"Site 31/6, Baikonur Cosmodrome,
Kazakhstan\",\n  \"SLC-3E, Vandenberg AFB, California, USA\"\n
],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n
}\n  },\n  {\n  \"column\": \"Date\",\n  \"properties\":
{\n  \"dtype\": \"string\",\n  \"num_unique_values\":
963,\n  \"samples\": [\n  \"Fri May 21, 2010 22:01
UTC\",\n  \"Fri Jan 15, 2016 16:57 UTC\",\n  \"Wed Sep
30, 2015 20:30 UTC\"\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Detail\",\n  \"properties\": {\n  \"dtype\": \"string\",\n
\"num_unique_values\": 962,\n  \"samples\": [\n  \"PSLV-
CA | TeLEOS-1 & Rideshares\",\n  \"GSLV Mk III | Demo Flight
(CARE)\",\n  \"Atlas V 421 | Morelos-3\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Rocket_Status\",\n
\"properties\": {\n  \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n  \"samples\": [\n
\"StatusRetired\",\n  \"StatusActive\"\n  ],\n
```

\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 288.45073217474595,\n        \"min\": 5.3,\n        \"max\": 5000.0,\n        \"num_unique_values\": 56,\n        \"samples\": [\n          50.0,\n          48.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Mission_Status\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Failure\",\n          \"Prelaunch Failure\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Rocket_Status_numerical\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"country\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"China\",\n          \"Russian Federation\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"country_alpha_3\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"CHN\",\n          \"RUS\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Year\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 12,\n        \"min\": 1964,\n        \"max\": 2020,\n        \"num_unique_values\": 50,\n        \"samples\": [\n          2007,\n          1981\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Month\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 12,\n        \"samples\": [\n          \"Oct\",\n          \"Nov\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe","variable_name":"df_data"}

```
rel_df = rel_df.drop_duplicates()
```

```
rel_df
```

{"summary":"{\n  \"name\": \"rel_df\",\n  \"rows\": 916,\n  \"fields\": [\n    {\n      \"column\": \"Organisation\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 25,\n        \"samples\": [\n          \"Virgin Orbit\",\n          \"Kosmotras\",\n          \"SpaceX\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 294.47044360188136,\n        \"min\": 5.3,\n        \"max\": 5000.0,\n

\"num_unique_values\": 56,\n        \"samples\": [\n          50.0,\n
48.5,\n          45.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Rocket_Status_numerical\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          0,\n          1\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"country_alpha_3\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 8,\n
\"samples\": [\n          \"CHN\",\n          \"RUS\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Year\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 12,\n        \"min\": 1964,\n
\"max\": 2020,\n        \"num_unique_values\": 50,\n
\"samples\": [\n          2007,\n          1981\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Month\",\n      \"properties\": {\
n        \"dtype\": \"category\",\n        \"num_unique_values\": 12,\
n        \"samples\": [\n          \"Oct\",\n          \"Nov\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Mission_Status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n
\"Failure\",\n          \"Prelaunch Failure\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"rel_df"}

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

ct =
ColumnTransformer(transformers=[(('one_hot_encoder'),OneHotEncoder(spa
rse_output=False),
['Organisation','country_alpha_3','Month'])],remainder='passthrough')

ct.fit(rel_df)

encoded_col = ct.get_feature_names_out()

print(encoded_col)
```

```
['one_hot_encoder__Organisation_Arianespace'
 'one_hot_encoder__Organisation_Boeing'
 'one_hot_encoder__Organisation_CASC'
'one_hot_encoder__Organisation_EER'
 'one_hot_encoder__Organisation_ESA'
 'one_hot_encoder__Organisation_Eurockot'
 'one_hot_encoder__Organisation_ExPace'
 'one_hot_encoder__Organisation_ILS'
```

```
'one_hot_encoder__Organisation_ISRO'
 'one_hot_encoder__Organisation_JAXA'
 'one_hot_encoder__Organisation_Kosmotras'
 'one_hot_encoder__Organisation_Lockheed'
 'one_hot_encoder__Organisation_MHI'
 'one_hot_encoder__Organisation_Martin Marietta'
 'one_hot_encoder__Organisation_NASA'
 'one_hot_encoder__Organisation_Northrop'
 'one_hot_encoder__Organisation_RVSN USSR'
 'one_hot_encoder__Organisation_Rocket Lab'
 'one_hot_encoder__Organisation_Roscosmos'
 'one_hot_encoder__Organisation_Sandia'
 'one_hot_encoder__Organisation_SpaceX'
 'one_hot_encoder__Organisation_ULA'
 'one_hot_encoder__Organisation_US Air Force'
 'one_hot_encoder__Organisation_VKS RF'
 'one_hot_encoder__Organisation_Virgin Orbit'
 'one_hot_encoder__country_alpha_3_CHN'
 'one_hot_encoder__country_alpha_3_FRA'
 'one_hot_encoder__country_alpha_3_IND'
 'one_hot_encoder__country_alpha_3_JPN'
 'one_hot_encoder__country_alpha_3_KAZ'
 'one_hot_encoder__country_alpha_3_NZL'
 'one_hot_encoder__country_alpha_3_RUS'
 'one_hot_encoder__country_alpha_3_USA' 'one_hot_encoder__Month_Apr'
 'one_hot_encoder__Month_Aug' 'one_hot_encoder__Month_Dec'
 'one_hot_encoder__Month_Feb' 'one_hot_encoder__Month_Jan'
 'one_hot_encoder__Month_Jul' 'one_hot_encoder__Month_Jun'
 'one_hot_encoder__Month_Mar' 'one_hot_encoder__Month_May'
 'one_hot_encoder__Month_Nov' 'one_hot_encoder__Month_Oct'
 'one_hot_encoder__Month_Sep' 'remainder__Price'
 'remainder__Rocket_Status_numerical' 'remainder__Year'
 'remainder__Mission_Status']
```

```
poping = ct.transform(rel_df)

ogog = pd.DataFrame(poping,columns = encoded_col)

from sklearn.preprocessing import LabelEncoder

lc = LabelEncoder()

snake = lc.fit_transform(ogog['remainder__Mission_Status'])

ogog['remainder__Mission_Status'] = snake

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

X = ogog.drop(['remainder__Mission_Status'],axis=1)
```

```
Y = ogog['remainder__Mission_Status']


sc = MinMaxScaler()
X = sc.fit_transform(X)
```

```
lc.classes_
```

```
array(['Failure', 'Partial Failure', 'Prelaunch Failure', 'Success'],
      dtype=object)
```

```
X
```

```
array([[0.        , 0.        , 0.        , ..., 0.00894949, 1.
,
        1.        ],
       [0.        , 0.        , 1.        , ..., 0.00489519, 1.
,
        1.        ],
       [0.        , 0.        , 0.        , ..., 0.01195267, 1.
,
        1.        ],
       ...,
       [0.        , 0.        , 0.        , ..., 0.01159829, 0.
,
        0.01785714],
       [0.        , 0.        , 0.        , ..., 0.01159829, 0.
,
        0.        ],
       [0.        , 0.        , 0.        , ..., 0.01159829, 0.
,
        0.        ]])
```

```
from sklearn.model_selection import train_test_split,GridSearchCV
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB,MultinomialNB
from sklearn.tree import DecisionTreeClassifier


X_train,X_test,y_train,y_test = train_test_split(X,Y,random_state=2 ,
test_size=0.2)

model_parms = {
    'svm': {
        'model':SVC(gamma='auto'),
        'params':{
            'C':[1,10,20],
            'kernel':['rbf','linear']
        }
```

```
    },
    'random_forest': {
        'model':RandomForestClassifier(),
        'params':{

            'n_estimators':[1,5,10]
        }
    },
    'logistic_regression': {
        'model':LogisticRegression(solver = 'liblinear',multi_class =
'auto'),
        'params':{
            'C':[1,5,10]
        }
    },
    'lgaussian': {
        'model':GaussianNB(),
        'params':{

        }
    },
    'multimodal': {
        'model':MultinomialNB(),
        'params':{

        }
    },
    'decision_tree': {
        'model':DecisionTreeClassifier(),
        'params':{
            'criterion': ['gini','entropy'],
        }
    }
}

scores = []

for m,k in model_parms.items():
  clf = GridSearchCV(k['model'],k['params'],cv=5,return_train_score =
False)
  clf.fit(X_train,y_train)

scores.append({'MODEL:':m,"BEST_SCORE:":clf.best_score_,"BEST_PARAMS":
clf.best_params_})
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/
_split.py:805: UserWarning:

The least populated class in y has only 1 members, which is less than
n_splits=5.
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split
.py:805: UserWarning:

The least populated class in y has only 1 members, which is less than
n_splits=5.

/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split
.py:805: UserWarning:

The least populated class in y has only 1 members, which is less than
n_splits=5.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:
```

```
'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
```

```
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:1247: FutureWarning:

'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to its
default value to avoid this warning.

/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split
.py:805: UserWarning:

The least populated class in y has only 1 members, which is less than
n_splits=5.

/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split
.py:805: UserWarning:

The least populated class in y has only 1 members, which is less than
n_splits=5.

/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split
.py:805: UserWarning:

The least populated class in y has only 1 members, which is less than
n_splits=5.
```

```
[print(f"{i['MODEL:']}          {i['BEST_SCORE:']}") for i in scores]
```

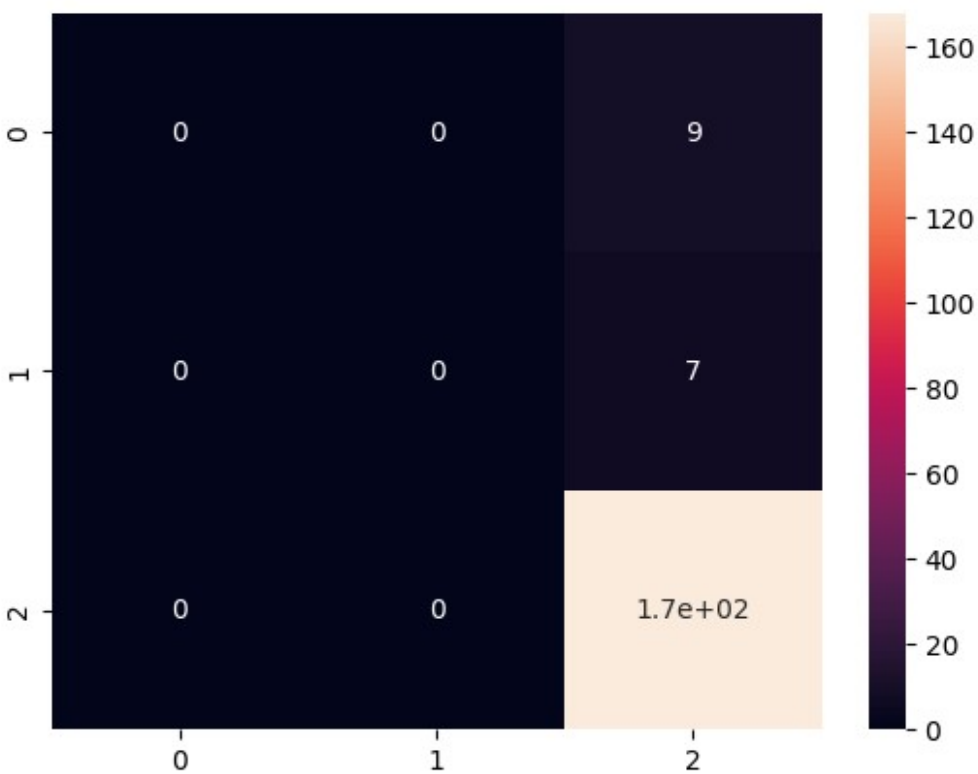
```
svm          0.9480943062156371
random_forest          0.9398751281334452
logistic_regression          0.9480943062156371
lgaussian          0.44533594259621656
multimodal          0.9467337619979499
decision_tree          0.9139222812412637
```

```
[None, None, None, None, None, None]
```

#CHOOSING SVM

```
from sklearn.metrics import
confusion_matrix,RocCurveDisplay,PrecisionRecallDisplay,classification
_report,accuracy_score,precision_score,r2_score

import seaborn as sns


model = SVC()

model.fit(X_train,y_train)

pred = model.predict(X_test)

co = confusion_matrix(y_test,pred)

sns.heatmap(co,annot=True)
```

```
<Axes: >
```




```
classification_report(y_test,pred)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this
behavior.

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this
behavior.

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this
behavior.
```

{"type":"string"}

```
accuracy_score(y_test,pred)
```

```
0.9130434782608695
```

```
from sklearn.metrics import precision_score, recall_score, f1_score
```

```
precision_score(y_test, pred, average='macro')
```

```
f1_score(y_test, pred, average='macro')
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this
behavior.
```

```
0.3181818181818182
```

```
recall_score(y_test,pred, average='macro')
```

```
0.3333333333333333
```

```
precision_score(y_test, pred, average='macro')
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning:
```

```
Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this
behavior.
```

```
0.3043478260869565
```

```
import joblib

joblib.dump(model,"snakes")
```

```
['snakes']
```